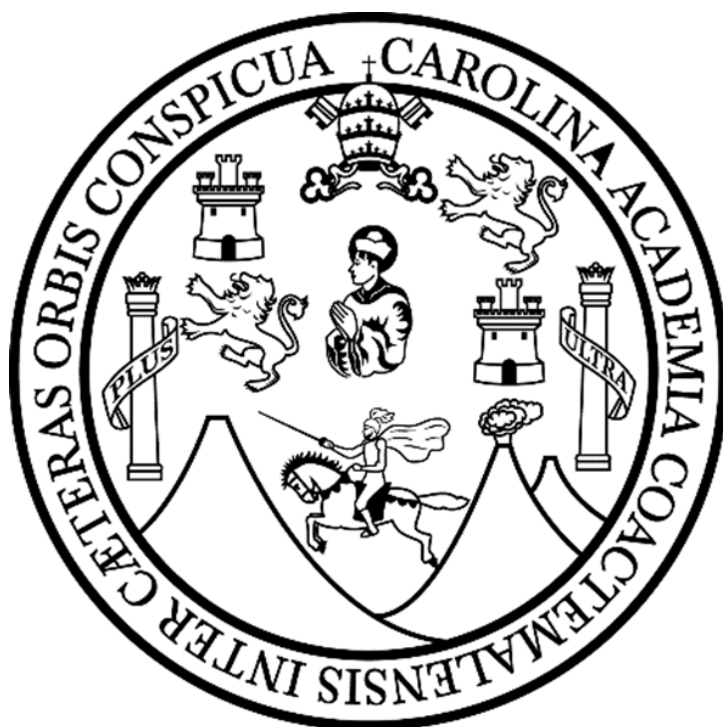


UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Facultad de Ingeniería
Escuela de Ciencias y Sistemas



Proyecto - Fase 2 Inteligencia Artificial

G#7

NOMBRE	CARNET
Vania Argueta Rodríguez	201213487
Henry Ronely Mendoza Aguilar	202004810
David Eduardo López Morales	201907483
Vernik Carlos Alexander Yaxon Ortiz	201712057

Manual Técnico

Introducción

Este manual describe en detalle la arquitectura, herramientas y metodologías utilizadas en el desarrollo del proyecto para el curso de Inteligencia Artificial 1 en la Universidad de San Carlos de Guatemala. El propósito del proyecto es implementar un modelo de inteligencia artificial capaz de interpretar entradas de texto en español y responder de forma coherente. Además, incluye la documentación necesaria para la instalación, configuración, y funcionamiento del sistema.

Objetivos

Desarrollar un modelo que interprete entradas de texto en idioma español y genere respuestas coherentes. Este modelo debe ejecutarse completamente en el navegador, sin depender de servicios externos.

Herramientas utilizadas

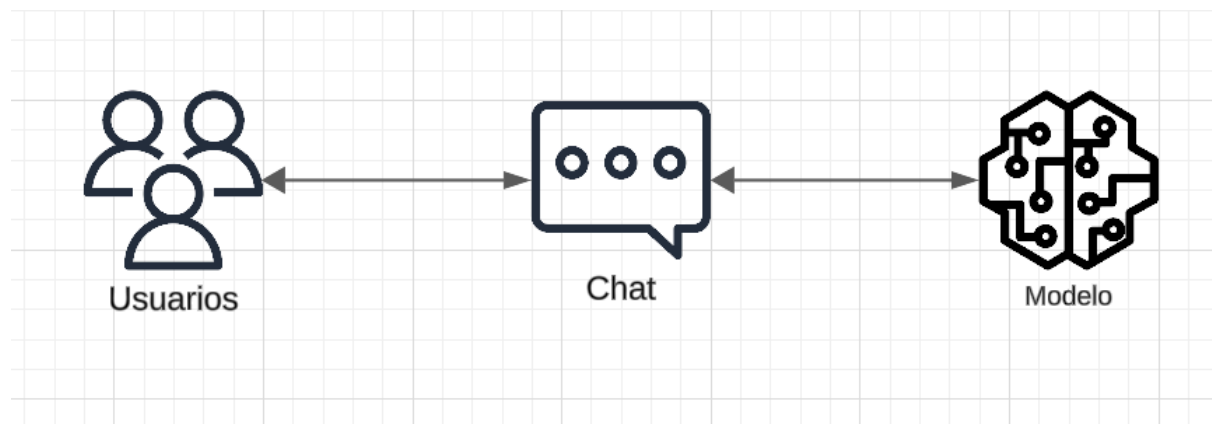
- **React + Vite:** React es una biblioteca de JavaScript para construir interfaces de usuario. En este proyecto, se utilizó junto con Vite, una herramienta de desarrollo que optimiza el tiempo de construcción y mejora el rendimiento del entorno de desarrollo. La combinación de estas herramientas permitió un desarrollo rápido y eficiente de componentes modulares y reutilizables.
- **Bootstrap:** Bootstrap es una biblioteca de diseño CSS que facilita la creación de interfaces responsive y estéticamente atractivas. En el proyecto, se usó para diseñar una interfaz de usuario que sea intuitiva y adaptable a diferentes dispositivos.
- **JavaScript:** El lenguaje principal del proyecto, JavaScript, permitió la implementación de la lógica de negocio y la interacción entre el modelo de inteligencia artificial y la interfaz gráfica. Su uso garantizó una integración fluida con las bibliotecas React y TensorFlow.js.
- **TensorFlow.js:** TensorFlow.js es una biblioteca para el desarrollo y entrenamiento de modelos de machine learning directamente en el navegador. En este proyecto, se utilizó para crear un modelo de red neuronal capaz de procesar texto en español y generar respuestas coherentes.

Arquitectura de la aplicación

El sistema tiene una arquitectura basada en tres capas principales:

1. **Capa de Presentación:** Diseñada con React y estilizada con Bootstrap para proporcionar una experiencia de usuario fluida y atractiva.
2. **Capa de Procesamiento:** Implementada con TensorFlow.js para procesar y generar respuestas basadas en entradas de texto.
3. **Capa de Interacción:** Gestionada por JavaScript para vincular la lógica del modelo con la interfaz gráfica del chat.

Flujo de interacción



Este diagrama representa el flujo de interacción en el sistema de chat basado en el modelo previamente entrenado

1. Los Usuarios envían un mensaje al sistema a través del Chat.
2. El Chat pasa la consulta al Modelo para su procesamiento.
3. El Modelo devuelve la respuesta más relevante al Chat.
4. Finalmente, el Chat presenta la respuesta generada a los usuarios.

Implementación del Modelo

El modelo de inteligencia artificial utilizado en este proyecto se basa en el Universal Sentence Encoder (USE) proporcionado por TensorFlow.js. Este modelo preentrenado permite codificar oraciones en vectores numéricos que luego se pueden utilizar para realizar tareas de procesamiento de lenguaje natural, como la clasificación o búsqueda de similitudes. Se implementó el modelo en Python y para ser más precisos utilizamos un cuaderno de Collab para poder aportar ideas todos.

Importación de Librerías Necesarias

```
!pip install tensorflowjs
```

```
from ast import pattern
import json
import string
import random
import pandas as pd
import numpy as np
import tensorflowjs as tfjs
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from keras.layers import Dense, Embedding, LSTM, Input, Flatten
from keras.models import Model
from sklearn.preprocessing import LabelEncoder
```

Definición de una Base de Conocimiento: Una lista de respuestas predefinidas se almacena en un array, y estas respuestas se comparan con la entrada del usuario:

```

"intents": [
  {
    "tag": "greetings",
    "patterns": [
      "Hi",
      "Hello",
      "Hey",
      "Hola",
      "\u00bfC\u00f3mo est\u00e9s?",
      "Buenos d\u00edas",
      "Buenas tardes",
      "Buenas noches"
    ],
    "responses": [
      "\u00a1Hola! \u00bfC\u00f3mo est\u00e9s?",
      "Hola, \u00bfen qu\u00e9 puedo ayudarte?",
      "\u00a1Buenos d\u00edas! \u00bfC\u00f3mo puedo asistirte?",
      "\u00a1Buenas tardes! \u00bfC\u00f3mo va tu d\u00eda?",
      "\u00a1Buenas noches! \u00bfAlgo en lo que pueda ayudarte?"
    ]
  },
  {
    "tag": "goodbye",
    "patterns": [
      "Goodbye",
      "Bye",
      "Adi\u00f3s",
      "Hasta luego",
      "Nos vemos",
      "Chao",
      "Cuidate"
    ],
    "responses": [
      "\u00a1Adi\u00f3s! Que tengas un gran d\u00eda.",
      "Nos vemos pronto, cu\u00e9date.",
      "Hasta luego, espero verte pronto.",
      "\u00a1Chao! Vuelve cuando quieras."
    ]
  },
  {
    "tag": "thanks",
    "patterns": [
      "Thank you",
      "Thanks",
      "Gracias",
      "Muchas gracias",
      "Te lo agradezco"
    ],
    "responses": [
      "\u00a1De nada! Estoy aqu\u00ed para ayudarte.",
      "No hay de qu\u00e9.",
      "\u00a1Con gusto!",
      "\u00a1Es un placer ayudarte!"
    ]
  }
]

```

Carga del Modelo y Generación de Respuestas: El modelo Universal Sentence Encoder se carga de forma asíncrona. Una vez cargado, codifica las oraciones de entrada y las respuestas para comparar similitudes:

```

with open('intents.json') as content:
    data = json.load(content)

tags = []
patterns = []
responses = {}

for intent in data['intents']:
    responses[intent['tag']] = intent['responses']

    for line in intent['patterns']:
        patterns.append(line)
        tags.append(intent['tag'])

data = pd.DataFrame({"patterns":patterns, "tags":tags})
data['patterns'] = data['patterns'].apply(lambda wrd:[ltrs.lower() for ltrs in wrd if ltrs not in string.punctuation])
data['patterns'] = data['patterns'].apply(lambda wrd: ''.join(wrd))

tokenizer = Tokenizer(num_words=2000)
tokenizer.fit_on_texts(data['patterns'])
train = tokenizer.texts_to_sequences(data['patterns'])

x_train = pad_sequences(train)
le = LabelEncoder()
y_train = le.fit_transform(data['tags'])

input_shape = x_train.shape[1]

vocabulary = len(tokenizer.word_index)
output_lenght = le.classes_.shape[0]

i = Input(shape=(input_shape,))
x = Embedding(vocabulary+1,10)(i)
x = LSTM(10,return_sequences=True)(x)
x = Flatten()(x)
x = Dense(output_lenght,activation='softmax')(x)
model = Model(i,x)

```

Guardamos el modelo generado y entrenado en python como un archivo tensorflow con estructura json y luego cargarlo en la parte de javascript.

```

model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model.fit(x_train, y_train, epochs=200)

print(model.summary())
#tfjs.converters.save_keras_model(model, '.')
tfjs.converters.save_keras_model(model, './modelo_python')
with open('tokenizer.json', 'w') as f:
    json.dump(json.loads(tokenizer.to_json()), f)

```

Carga del Modelo Generado en Python: Ahora luego de tener el modelo entrenado y compilado en python procedemos a cargarlo en memoria desde JavaScript con React.

```
// Cargar el modelo y el tokenizador al iniciar el componente
useEffect(() => {
  const loadModelAndTokenizer = async () => {
    try {
      console.log("Cargando modelo...");
      model = await tf.loadLayersModel("/modelo/modelo.json"); // Ruta al modelo
      console.log("Modelo cargado correctamente.");
    } catch (error) {
      console.error("Error al cargar el modelo:", error);
    }

    try {
      console.log("Cargando tokenizer...");
      const tokenizerResponse = await fetch("/modelo/tokenizer.json"); // Ruta al tokenizador
      tokenizer = await tokenizerResponse.json();
      console.log("Tokenizer cargado correctamente.");
    } catch (error) {
      console.error("Error al cargar el tokenizer:", error);
    }
  };
});
```

Generamos un método para obtener las respuestas del modelo ya cargado según los inputs que el usuario genere.

```
// Obtener respuesta del modelo
const getResponse = async (inputText) => {
  if (!model || !tokenizer) {
    console.error("Modelo o tokenizador no cargado.");
    return "Error al cargar el modelo.";
  }

  const sequences = tokenizeInput(inputText); // Tokenizar entrada
  const paddedSequences = padSequences(sequences, model.inputs[0].shape[1]); // Padding
  const prediction = model.predict(tf.tensor2d(paddedSequences)); // Predicción

  // Extraer la respuesta generada (si es texto)
  const response = await prediction.data(); // Obtiene las probabilidades o texto generado
  return response.join(" "); // Ajusta según el formato de tu modelo
};
```

Y por último generamos una respuesta predictiva según el input del usuario.

```
try {
  const botResponse = await getResponse(input);
  const botMessage = { text: botResponse, sender: "bot", timestamp };
  setMessages((prevMessages) => [...prevMessages, botMessage]);
} catch (error) {
  const errorMessage = {
    text: "Ocurrió un error al procesar tu mensaje. Intenta nuevamente.",
    sender: "bot",
    timestamp,
  };
  setMessages((prevMessages) => [...prevMessages, errorMessage]);
  console.error("Error al predecir:", error);
}
```